

Análise empírica de A* versus Dijkstra para menor caminho em grids 2D com obstáculos

Fábio Krein

Programa de Pós-Graduação em Computação Aplicada — Unisinos

Disciplina: Análise de Algoritmos

Resumo

Este trabalho apresenta uma análise empírica comparativa entre os algoritmos Dijkstra e A* (com heurística de Manhattan) para o problema de menor caminho em grids bidimensionais com obstáculos. Conduzimos 540 execuções variando tamanho do mapa (50×50 , 100×100 , 200×200) e densidade de obstáculos (10%, 20%, 30%), com 30 sementes por configuração. Os resultados confirmam que ambos produzem caminhos de custo idêntico, mas o A* expande até cinco vezes menos nós no cenário mais denso. Um teste de Wilcoxon pareado mostra que todas as diferenças de tempo são estatisticamente significativas ($p \leq 0,001$): o ganho é expressivo em mapas grandes e densos ($\approx 3,8\times$ em 200×200 com 30% de obstáculos), porém pequeno e negativo em mapas esparsos, em que o sobrecusto por nó da heurística não é amortizado.

1. Introdução

O cálculo de menor caminho em grafos é um problema fundamental em ciência da computação, com aplicações em navegação de robôs, videogames e sistemas de roteamento. O algoritmo de Dijkstra [3] garante a otimalidade da solução em grafos com pesos não negativos por meio de uma busca uniforme a partir da origem, mas expande nós sem qualquer informação sobre a posição do alvo — o que tende a tornar a busca cara em instâncias grandes. Hart, Nilsson e Raphael [1] propuseram o A* como uma generalização do Dijkstra capaz de incorporar conhecimento de domínio via uma função heurística $h(n)$ que estima o custo remanescente até o objetivo; quando h é admissível, a otimalidade é preservada e o número de nós expandidos tende a diminuir. Trabalhos aplicados recentes [2] seguem revisitando essa comparação em cenários reais.

O objetivo deste trabalho é reproduzir empiricamente, em um cenário próprio — grids 2D com obstáculos aleatórios —, o ganho previsto pela formulação de [1] e quantificar em que condições o A* efetivamente compensa o sobrecusto por nó da heurística. Especificamente, investigamos duas questões: (i) o A* com uma heurística admissível preserva a otimalidade do Dijkstra neste cenário? e (ii) sob quais condições de tamanho e densidade a redução no número de nós expandidos se traduz em redução efetiva do tempo de execução? Comparamos as duas estratégias quanto a custo do caminho, número de nós visitados e tempo de execução, variando sistematicamente o tamanho do mapa e a densidade de obstáculos.

2. Artigo selecionado e algoritmo estudado

O artigo principal adotado como referência é o trabalho seminal de Hart, Nilsson e Raphael [1], que introduz o A* e fornece a base formal para a determinação heurística de caminhos de custo mínimo. Como referência complementar utilizamos Ardiansyah et al. [2], que compara Dijkstra e A* para roteamento em malhas viárias.

O A* é uma busca em grafo informada por uma função $f(n) = g(n) + h(n)$, em que $g(n)$ é o custo acumulado da origem até n e $h(n)$ é uma estimativa do custo de n até o objetivo. A cada iteração o algoritmo remove da fila de prioridades o nó com menor f e expande seus vizinhos. Hart et al. demonstram que, se h é admissível ($h(n) \leq h^*(n)$ para todo n), o A* retorna um caminho ótimo; quando h é, além disso, consistente, cada nó é expandido no máximo uma vez. O Dijkstra corresponde ao caso $h \equiv 0$: a busca passa a ser uniforme. Em um grid 4-conectado com custos unitários, a distância de Manhattan $h(n) = |x_n - x_a| + |y_n - y_a|$ é admissível e consistente — qualquer caminho real entre dois pontos respeita ao menos essa quantidade de passos — sendo portanto a escolha natural para o cenário avaliado.

Implementados com fila de prioridade baseada em heap binário, ambos os algoritmos têm complexidade de pior caso $O((V + E) \log V)$, onde V e E são os números de vértices e arestas; em um grid $n \times n$ 4-conectado, $V = n^2$ e $E = O(n^2)$. O A* compartilha esse limite assintótico com o Dijkstra — a vantagem é de fator constante e de caso médio, ao reduzir o número de vértices efetivamente expandidos por meio da poda guiada pela heurística. É exatamente essa redução, e sua conversão (ou não) em tempo, que este trabalho quantifica empiricamente.

3. Implementação e metodologia experimental

3.1 Implementação

Ambos os algoritmos foram implementados em Python 3, sem bibliotecas externas de grafos ou pathfinding. Compartilham a mesma estrutura: fila de prioridade baseada em heap binário (heapq), dicionário de custos acumulados g e conjunto de nós já expandidos. A única diferença é a chave de prioridade — Dijkstra usa $g(n)$ e A* usa $f(n) \coloneqq g(n) + h(n)$. Para tornar a comparação justa, os dois retornam a mesma estrutura de saída e consultam a vizinhança pela mesma função auxiliar. O grid é uma matriz $n \times n$ com 0 indicando célula livre e 1, obstáculo; a vizinhança é 4-conectada e o custo de cada aresta é unitário. A origem (0, 0) e o destino ($n-1$, $n-1$) são forçados a permanecerem livres.

3.2 Cenário experimental e métricas

Os experimentos cobrem o produto cartesiano de três tamanhos (50×50, 100×100, 200×200) e três densidades de obstáculos (10%, 20%, 30%). Cada uma das nove configurações foi executada com 30 sementes (0 a 29), totalizando 540 execuções considerando os dois algoritmos. As sementes garantem reprodutibilidade e fazem com que Dijkstra e A* sejam avaliados sobre os mesmos mapas. Como os obstáculos são sorteados de forma independente, densidades altas podem produzir grids sem caminho viável entre origem e destino — um efeito de percolação: a fração de instâncias solúveis cai de 100% (90/90) a 10% de obstáculos para 86% (77/90) a 20% e apenas 47% (42/90) a 30%, sem concentração em um tamanho específico. As 122 execuções sem caminho (61 mapas $\times$ 2 algoritmos) foram excluídas das estatísticas de desempenho: nesses casos ambos os algoritmos varrem exaustivamente toda a componente livre acessível e o A* não tem qualquer vantagem, pois a poda heurística só atua quando existe um objetivo alcançável.

Foram registradas três métricas: (i) custo do caminho em número de passos, que serve simultaneamente como qualidade da solução e verificação de sanidade; (ii) número de nós visitados, definido como a cardinalidade do conjunto de nós removidos da fila de prioridade — uma medida determinística, independente da máquina; (iii) tempo de execução, medido com `time.perf_counter` em milissegundos. Como uma única cronometragem é sensível a ruído de escalonamento do sistema operacional e a aquecimento de cache/interpretador, cada instância foi executada dez vezes e reportamos a mediana dos tempos; todas as medições foram feitas em um único host, em Python 3. Para cada configuração reportamos média e desvio padrão sobre as sementes com caminho viável.

4. Resultados

A Tabela 1 sintetiza os valores médios obtidos para as três métricas, e as Figuras 1 a 3 mostram graficamente o comportamento em função do tamanho do mapa, para cada densidade de obstáculos.

Tamanho	Obst.	Nós visitados (méd.)		Tempo ms (méd.)		Razão A*/Dij
		Dijkstra	A*	Dijkstra	A*	(tempo)
50	10%	2.254	1.971	2,23	2,46	1,11×
50	20%	2.000	1.277	1,93	1,59	0,83×
50	30%	1.706	668	1,60	0,79	0,50×
100	10%	8.998	7.792	9,47	10,40	1,10×
100	20%	7.978	4.940	8,27	6,77	0,82×
100	30%	6.822	1.941	6,95	2,48	0,36×
200	10%	35.986	31.193	41,18	47,42	1,15×
200	20%	31.909	19.484	35,86	30,51	0,85×
200	30%	27.417	5.479	30,52	8,10	0,27×

Tabela 1. Resultados médios por configuração (médias sobre as sementes com caminho viável). Custos de caminho omitidos por serem idênticos entre os dois algoritmos. Razões < 1 indicam vantagem do A*; todas as diferenças de tempo são estatisticamente

significativas (Wilcoxon pareado, $p \leq 0,001$).

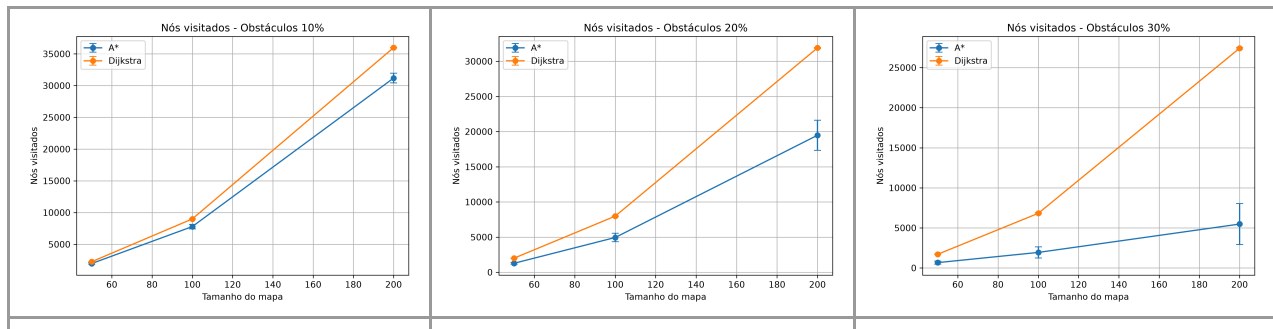


Figura 1. Nós visitados em função do tamanho do mapa para densidades de 10%, 20% e 30% de obstáculos (média $\pm$ desvio padrão sobre 30 sementes).

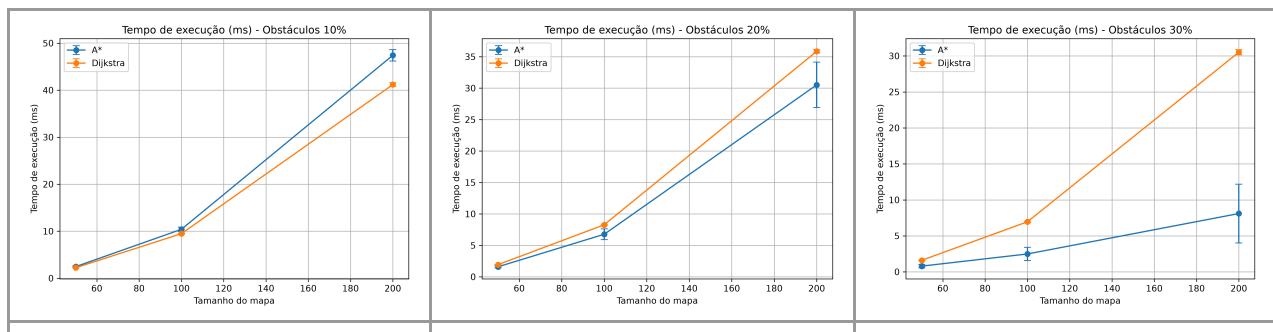


Figura 2. Tempo de execução (ms) em função do tamanho do mapa para densidades de 10%, 20% e 30% de obstáculos.

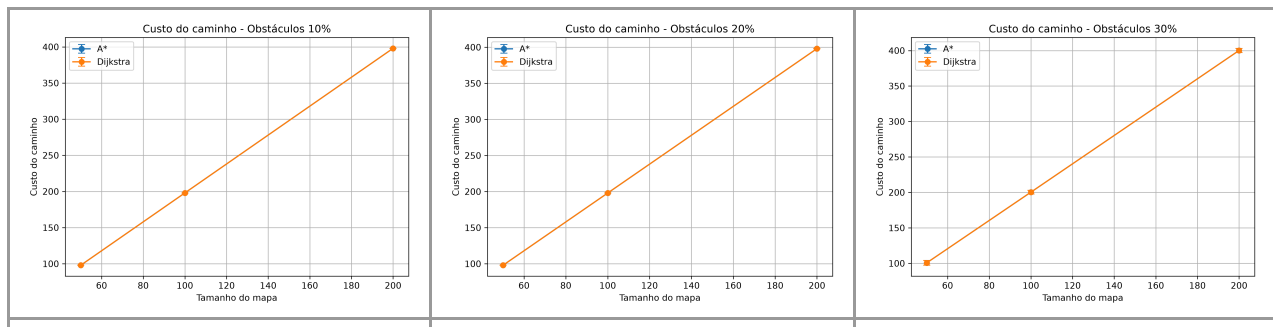


Figura 3. Custo do caminho encontrado em função do tamanho do mapa. Os pontos de Dijkstra e A* se sobrepõem em todas as configurações.

4.1 Custo do caminho

O custo do caminho retornado pelos dois algoritmos foi idêntico em todas as 418 execuções com caminho viável (Figura 3). As curvas de Dijkstra e A* coincidem ponto a ponto e o desvio padrão por configuração é igual entre as duas séries, o que confirma empiricamente a otimalidade prevista pela teoria: como a distância de Manhattan é admissível em grids 4-conectados, o A* preserva a garantia do Dijkstra de retornar o menor caminho.

4.2 Nós visitados

A diferença mais expressiva entre os dois algoritmos aparece no número de nós expandidos (Figura 1). Em grids esparsos (10% de obstáculos) o A* visita cerca de 87% dos nós do Dijkstra — uma redução modesta, pois com poucos obstáculos a heurística raramente consegue podar exploração relevante. A vantagem cresce de forma consistente com a densidade: a 20% o A* expande aproximadamente 60% dos nós, e a 30% essa fração cai para 39% (50×50), 28% (100×100) e apenas 20% no maior grid. O caso mais marcante é o 200×200 com 30% de obstáculos, em que o A* visita em média 5.479 nós contra 27.417 do Dijkstra. Observa-se também que o desvio padrão do A* é muito maior que o do Dijkstra, refletindo a sensibilidade da heurística ao arranjo específico dos obstáculos.

4.3 Tempo de execução

O tempo de execução acompanha qualitativamente a curva de nós visitados, porém com inclinação menos pronunciada (Figura 2). Em 200×200 com 30% de obstáculos o A* leva em média 8,1 ms contra 30,5 ms do Dijkstra — uma aceleração de aproximadamente 3,8 vezes. Já em cenários com 10% de obstáculos o A* apresenta tempo médio ligeiramente superior ao do Dijkstra (entre 10% e 15%), apesar de visitar menos nós. Esse comportamento é discutido em detalhe na próxima seção.

5. Discussão e limitações

A equivalência de custos confirma empiricamente a otimalidade prevista em [1]: como $h \equiv 0$ é trivialmente admissível, Dijkstra é caso particular do A* e a consistência da Manhattan garante que nenhum nó seja reexpandido. A diferença em nós visitados tem origem geométrica: Dijkstra expande nós em ordem crescente de g , formando uma onda quase circular ao redor da origem; o A* prioriza $f \approx g + h$ e, em um grid livre, varre apenas o retângulo entre origem e objetivo. Com obstáculos, o A* só explora desvios cujo f não seja dominado pelo caminho direto, enquanto o Dijkstra continua expandindo radialmente. A área varrida pelo Dijkstra cresce com o quadrado do raio; o A* permanece confinado a um corredor cuja largura aumenta apenas com o número de desvios necessários — daí a queda monotônica da razão A*/Dijkstra em expansões à medida que a densidade aumenta. O alto desvio padrão do A* reflete a variância geométrica desses corredores.

5.1 Por que A* nem sempre é mais rápido

Embora o A* expanda menos nós em todas as configurações, sua vantagem em tempo só se materializa a partir de 20% de obstáculos. Cada expansão envolve cálculo da heurística, somas adicionais e chave de prioridade maior — cada nó custa mais ciclos do que no Dijkstra. Quando o ganho em poda é pequeno (grids esparsos, em que a Manhattan é quase exata e descarta poucos vizinhos), esse sobrecusto por nó não é amortizado e o tempo total ultrapassa o do baseline. O efeito é visível em 200×200 com 10%: uma redução de $\sim 13\%$ no número de nós visitados se converte em execução $\sim 15\%$ mais lenta. Para descartar que essa desvantagem fosse mero ruído de medição, cada instância foi cronometrada dez vezes (registrando-se a mediana) e os dois algoritmos foram comparados com um teste de Wilcoxon de postos sinalizados, pareado pela semente. Em todas as nove configurações a diferença de tempo é estatisticamente significativa ($p < 0,001$): o A* é significativamente mais lento nas três configurações com 10% de obstáculos e significativamente mais rápido nas seis com 20% e 30%. A desvantagem em cenários esparsos é, portanto, um efeito real e reproduzível, não artefato de variância. Ardiansyah et al. [2] reportam comportamento semelhante em malhas viárias — vantagens em expansões nem sempre se traduzem proporcionalmente em tempo.

5.2 Limitações e trabalhos futuros

Os experimentos consideram custo de aresta unitário e vizinhança 4-conectada; pesos heterogêneos (terreno com custos variáveis) ou movimentos diagonais exigiriam outra heurística — respectivamente uma heurística ponderada ou a distância octile/euclidiana — e alterariam o trade-off entre poda e sobrecusto por nó. Estender o estudo a esses cenários é a direção natural de adaptação do método a problemas de otimização mais próximos de aplicações reais. Os obstáculos são amostrados de forma independente, produzindo padrões mais ruidosos do que mapas reais — resultados em malhas viárias [2] podem diferir em magnitude. Não avaliamos consumo de memória, embora o tamanho máximo da fronteira (fila de prioridade) seja um eixo em que A* e Dijkstra também divergem e mereça medição futura. Por fim, os tempos absolutos dependem da máquina e do overhead do interpretador Python; por isso as conclusões se apoiam nas razões entre algoritmos e no número de nós expandidos, ambos robustos a esses fatores.

6. Conclusão

Reproduzimos empiricamente o resultado central de Hart, Nilsson e Raphael [1]: para menor caminho em grids 2D, o A* com heurística de Manhattan preserva a otimalidade do Dijkstra e expande sistematicamente menos nós, com vantagem crescente com a densidade — até cinco vezes menos expansões no cenário mais denso. A tradução desse ganho em tempo, contudo, depende de o sobrecusto por nó da heurística ser amortizado pela poda: em mapas grandes mas esparsos o A* é significativamente, ainda que modestamente, mais lento (10–

15%); a partir de 20% de obstáculos passa a ser significativamente mais rápido, chegando a $\approx 3,8\times$ em 200×200 com 30%. O experimento ilustra uma lição prática complementar à formulação teórica de [1]: o benefício de uma heurística admissível existe sempre em expansões, mas o ganho de tempo só se concretiza quando a estrutura do problema favorece a poda. Em aplicações com mapas densos e de larga escala, o A* mostra-se claramente preferível; estendê-lo a grafos ponderados e vizinhanças mais ricas é o próximo passo para aproximá-lo de cenários reais de otimização.

Referências

- [1] HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, v. 4, n. 2, p. 100–107, 1968.
- [2] ARDIANSYAH et al. Comparative analysis of Dijkstra and A* algorithms for determining the shortest route, 2025.
- [3] DIJKSTRA, E. W. A note on two problems in connexion with graphs. Numerische Mathematik, v. 1, p. 269–271, 1959.

Código-fonte e dados para reprodução disponíveis em: <https://github.com/valdomirosouza/astar-dijkstra-analysis>